F21SC Coursework 1

By: Mohamad Ajaz

H00458396

Introduction:

The purpose of this report is to provide a simple to follow guide on setting up Squash Browser.

Squash Browser is a simple and intuitive new way to browse on the web.It isn't just another chromium web client; it is a simple, intuitive, and radically new way to experience the web!

It rivals already existing heavy and clunky chromium web browsers such as Google Chrome, Opera, Edge,etc. Here at squash labs we revolutionized the way browsers work by not even including a web rendering engine! Crazy right? Not only does this make our browser the best performative browser in the market, there is absolutely no way for third-party trackers, pixels, or intrusive JavaScript to execute or be displayed in our application, meaning you get true, undeniable, and effortless privacy!

Squash Browser is not a bug, it's a feature. We're offering a direct, powerful, and text-only browsing experience.

The way the internet was always meant to be.

Assumptions:

- The user history list would not need to show the title for each history item
- A list was used as the data structure for the navigation stack as it allows forward and backward access without affecting the stack.
- The initial home page URL would initialize with “https://hw.ac.uk”, but the user would be allowed to change it to any URL as they please. (“https://hw.ac.uk” is just a suggestion for the user at the start of the browser)
- Navigation stack wouldn’t need to be saved in the database as it was not specified in the requirements. As such when the user loads the browser again only the the most recent history would be loaded.
- Using the alternative Avalonia was allowed by my professor, while mentioned in the spec that GTK# Library for linux must be used.

Feature checklist:

FEATURE	STATUS
Sending HTTP requests and displaying the contents of the response	Fully Implemented
Displaying the HTTP response status code	Fully Implemented
Displaying the web page’s title	Fully Implemented
Reload the current web page	Fully Implemented
Setting the home page	Fully Implemented

Adding pages to bookmarks list	Fully Implemented
Naming bookmarks	Fully Implemented
Deleting bookmarks	Fully Implemented
Navigating to bookmarks	Fully Implemented
Maintaining history	Fully Implemented
Navigating to previous/next page in history	Fully Implemented
Navigating to a selected page in history	Fully Implemented
List five urls from the web-page in the side panel	Fully Implemented
Overall good GUI experience	I dunno, yours to judge
Settings (home page, bookmarks, history) persist across sessions	Mostly implemented - As under my assumptions i assumed navigation history would not need to be saved.
Database used as storage	Fully Implemented
Advanced Added multi-user functionality.	Not Implemented

Design Considerations:

1. Architecture:

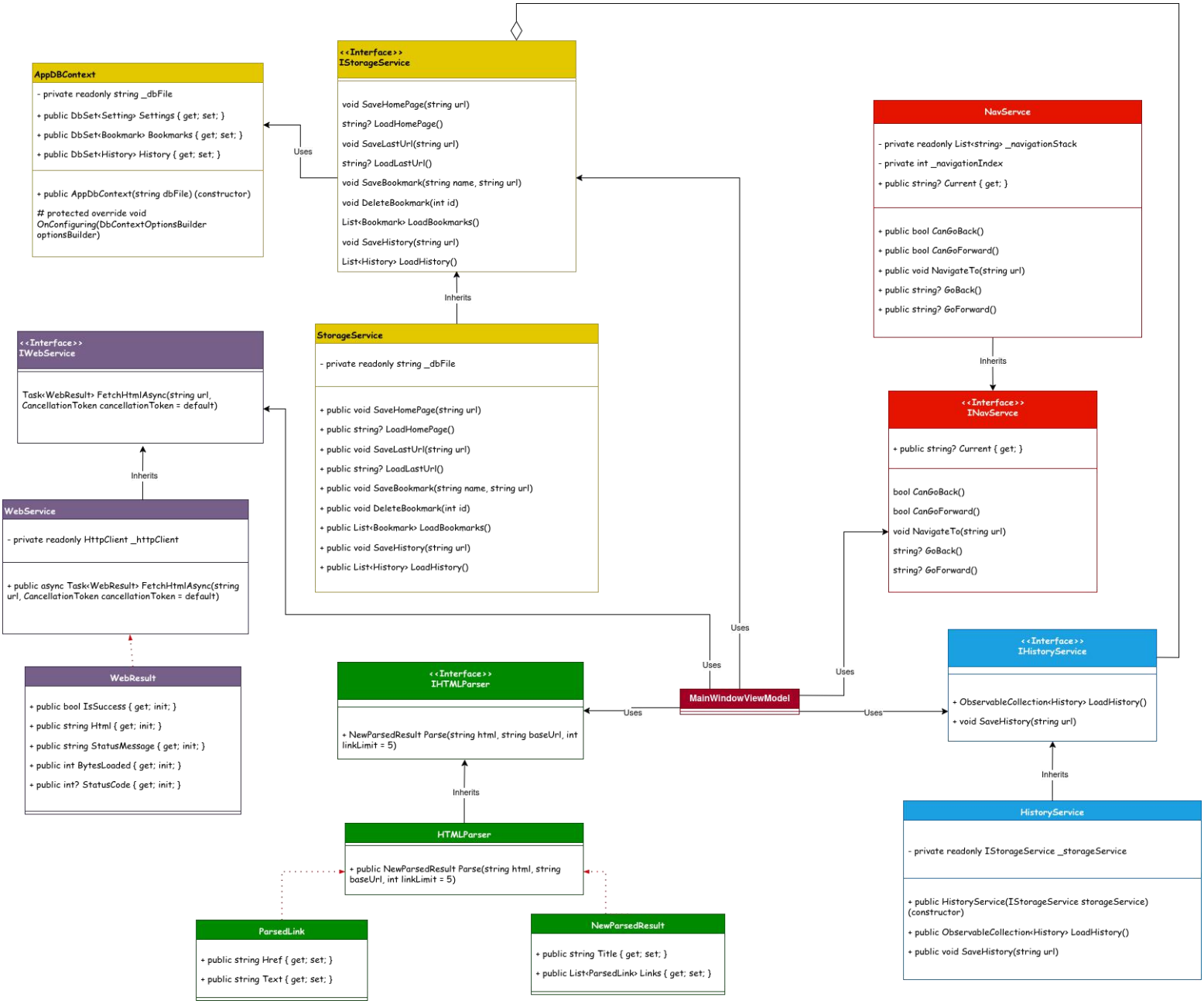
The architecture used in this project is the MVVM (Model, View, ViewModel).MVVM basically allows us to separate the GUI logic with the business/backend logic, effectively allowing me to maintain and scale the codebase in an easier fashion. It is also regarded as the go-to default architecture for many frameworks like WPF, Xamarin, etc.
Below is the file structure of the squash web browser project. Commands folder holds classes that are used to connect UI elements to the ViewModel logic. Controls folder holds custom reusable UI elements such as the address bar, bookmarks panel, etc. Models folder holds application's data classes. These classes represent the core objects of the application, such as a Bookmark or a History entry. Services holds the classes that provide core functionality for the app. Such as IWebService to make http requests add IStorageService that handles all the database logic. ViewModels folder holds the core business logic binding UI elements to specific functions from the Services folder to be

called. Finally views contains the main windows and pages UI of the application's user interface.

```
Squash_Web_Browser/
├── Commands/
│   └── RelayCommand.cs
│
├── Controls/
│   ├── AddressBar.axaml
│   ├── AddressBar.axaml.cs
│   ├── BookmarksPanel.axaml
│   ├── BookmarksPanel.axaml.cs
│   ├── EditHomePagePanel.axaml
│   ├── EditHomePagePanel.axaml.cs
│   ├── HistoryPanel.axaml
│   └── HistoryPanel.axaml.cs
│
├── Models/
│   ├── Bookmark.cs
│   ├── History.cs
│   └── Setting.cs
│
├── Services/
│   ├── AppDbContext.cs
│   ├── IHistoryService.cs
│   ├── IHtmlParser.cs
│   ├── INavService.cs
│   ├── IStorageService.cs
│   └── IWebService.cs
│
├── ViewModels/
│   ├── MainWindowViewModel.cs
│   └── ViewModelBase.cs
│
└── Views/
    ├── App.axaml
    ├── App.axaml.cs
    ├── MainWindow.axaml
    └── MainWindow.axaml.cs
```

Appendix 1 - Class diagrams:

This section covers the class diagram of the services folder, containing classes that handle most of the business logic such as the html parsing, database storing, url navigation, bookmark, history, etc.



Appendix 2 - Entity Relationship Diagram:

GUI Design & Advanced language constructs:

Architecture:

Developer Guide: